



MÉMOIRE DE VEILLE TECHNOLOGIQUE

Master 1 Lead-dev – Coding Factory by Esiee-IT

MVT

Internal Developer Platform

Comment construire une Internal Developer Platform qui réduit vraiment la friction du développement d'une application jusqu'à sa mise en production, sans pour autant devenir une charge bureaucratique ?

Réalisé par :

Tristan TURGY

Valdo MARGUERITE

Diego CARVALHO DOS SANTOS

27/06/2026

Sommaire

Sommaire	2
Introduction	4
Synthèse	6
Problématique	6
Hypothèses testées	6
Résultats	6
Recommandation	7
Contre-arguments	8
Tableaux de validation	10
Table des hypothèses	10
Table des claims → sources	10
Hypothèse 1 : une IDP bien conçue réduit-elle vraiment la friction dev→prod ?	13
Annonce et cadrage	13
I. Arguments	13
II. Contre-arguments et limites	14
III. Recommandation finale, décision Lead Developer	15
Hypothèse 2 : l'adoption d'une IDP dépend-elle davantage de facteurs organisationnels et culturels que technologiques ?	16
I. L'IDP comme réponse à un problème de complexité sociotechnique	16
II. La structure organisationnelle conditionne fortement l'adoption	17
III. L'adoption dépend principalement de la culture et de l'accompagnement humain	18
Recommandation, décision Lead Developer	19
Hypothèse 3 : les IDP open-source (Backstage) l'emportent-elles sur le custom en TCO et time-to-value pour une équipe réduite ?	20
Indicateurs et protocole de validation	20
I. Backstage : un écosystème mature qui apporte de la valeur vite	20
II. Le vrai coût du custom : un TCO souvent sous-estimé	21
III. Quand l'avantage de Backstage se nuance	22
Apport du podcast	22
Recommandation, décision Lead Developer	23
Conclusion	24

Décision	24
Plan de suite et critères de décision	24
Vocabulaire	26
Sources.....	28

Introduction

Les entreprises investissent massivement dans les outils cloud, l'automatisation et l'IA afin d'accélérer le processus de développement jusqu'à sa mise en production. Selon le cabinet de recherche Omdia, les investissements dans les infrastructures cloud ont augmenté de 110 milliards de dollars sur le dernier trimestre 2025 et de **400 milliards de dollars au total sur l'année**, représentant une augmentation de 29 % en un an **(1)**. Ces investissements ont permis aux développeurs de gagner du temps : **68 % des développeurs disent gagner plus de 10 h sur une semaine grâce aux outils IA** en 2025, selon le rapport *State of Developer Experience 2025* d'Atlassian **(2)**. En revanche, selon ce rapport, les développeurs ne consacrent **que 16 % de leur temps à coder**.

En dehors du temps de développement, comment les entreprises peuvent-elles accélérer le processus de développement ? Un grand axe d'amélioration se situe sur la manière dont l'entreprise s'organise. Toujours selon ce rapport d'Atlassian, **90 % des développeurs perdent 6 heures ou plus par semaine à cause de frictions organisationnelles**. De plus, la moitié des développeurs interrogés disent perdre plus de 10 heures. Cela provient notamment de la recherche d'information, de changements de contexte entre outils ou encore de la compréhension de processus internes.

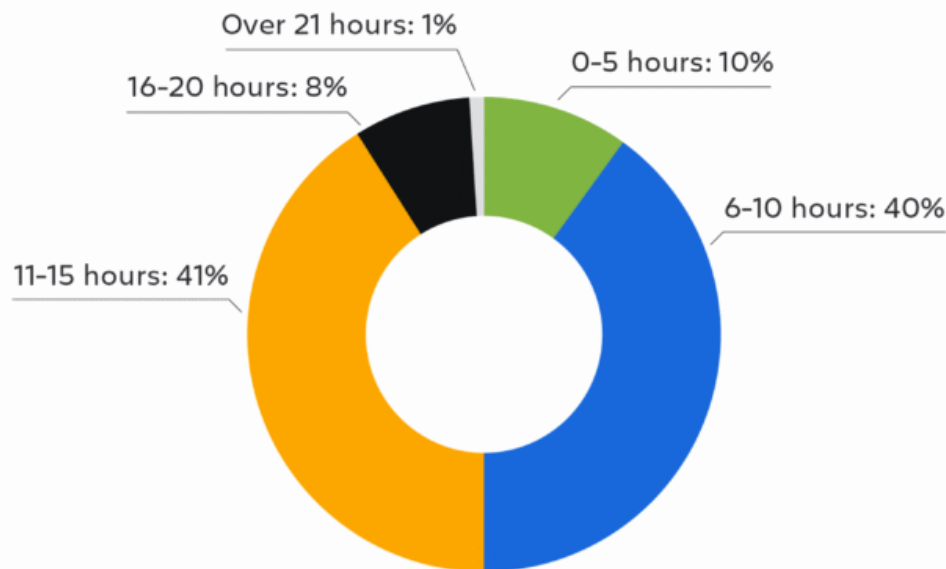


Figure : nombre d'heures perdues par semaine par les développeurs interrogés à cause de frictions organisationnelles.
Source : Atlassian, *State of Developer Experience 2025*.

Ce constat révèle un paradoxe : alors que les entreprises multiplient les outils censés améliorer le quotidien et la productivité des développeurs, **ces mêmes outils peuvent être la source de complexité ou de frictions**. Les développeurs passent plus de temps à gérer ces frictions qu'à produire du code, de la valeur métier.

Dans ce contexte, la discipline du Platform Engineering, qui consiste à construire et maintenir des plateformes internes pour améliorer la productivité des développeurs, propose des **Internal Developer Platforms (IDP)** comme moyen de **standardiser les processus de développement et de réduire les frictions entre le début du développement jusqu'à sa mise en production**. Ces IDP sont des plateformes (internes ou externes) permettant aux développeurs de créer et déployer des applications plus facilement, sans devoir gérer toute la complexité d'une infrastructure. Elles agissent comme des couches entre les développeurs et l'infrastructure (physique ou cloud). Par exemple, au lieu que chaque équipe configure Kubernetes, crée des pipelines CI/CD, etc., une IDP fournit des outils standardisés et automatisés pour réaliser toutes ces étapes. Cependant, une plateforme trop rigide ou abstraite pourrait rapidement devenir un **nouvel obstacle pour le développeur ou une charge bureaucratique**. L'IDP qui devait se transformer en une initiative pour simplifier le travail du développeur deviendrait une couche supplémentaire organisationnelle.

Par conséquent, comment construire une Internal Developer Platform qui réduit vraiment la friction du développement d'une application jusqu'à sa mise en production, sans pour autant devenir une charge bureaucratique ?

Afin d'apporter des éléments de réponse à cette problématique, ce mémoire de veille technologique s'appuie sur trois hypothèses. Dans un premier temps, on analysera si une **IDP bien conçue permet effectivement de réduire la friction du développement d'une application jusqu'à sa mise en production**, en se basant sur des indicateurs comme le Lead Time, le Mean Time To Recovery (MTTR) ou encore le taux d'échecs de déploiement (Change Failure Rate). Toutefois, les bénéfices d'une IDP ne reposent pas uniquement sur ses caractéristiques techniques. Dans un second temps, on se concentrera sur le contexte organisationnel dans lequel une IDP est déployée (réduction des silos, collaboration entre les différentes équipes), pour se demander si **l'adoption d'une IDP dépend davantage de facteurs organisationnels et culturels que de choix technologiques**. Enfin, dans une dernière hypothèse, on se demandera si **une IDP open-source comme Backstage réduit les coûts et accélère l'adoption par rapport à une plateforme entièrement développée en interne**. Cette analyse se basera sur les coûts, la maintenance, la viabilité et la pérennité grâce à leurs communautés.

Synthèse

Problématique

Les équipes produit perdent du temps sur des tâches d'infrastructure répétitives (provisioning, configuration de pipelines CI/CD, onboarding, conformité) faute d'un point d'entrée unifié. L'Internal Developer Platform est censée résoudre ce problème en standardisant ces processus et en exposant des capacités self-service aux développeurs. Mais mal conçue, elle devient une couche bureaucratique supplémentaire : des gates obligatoires, des formulaires, des approbations qui ralentissent plus qu'elles n'accélèrent.

La question centrale de ce mémoire est donc : *comment construire une IDP qui réduit vraiment la friction du développement jusqu'à sa mise en production, sans devenir une charge bureaucratique ?*

Hypothèses testées

Trois hypothèses structurent notre analyse.

- **H1** — Une IDP bien adoptée réduit la friction dev→prod, et cette réduction est mesurable via les métriques DORA : Lead Time for Changes (LTfC), Mean Time To Recovery (MTTR) et Change Failure Rate (CFR).
- **H2** — L'adoption d'une IDP dépend fortement de la culture organisationnelle : la présence ou l'absence de silos, la collaboration inter-équipes, et l'existence d'un champion interne sont des facteurs déterminants.
- **H3** — Les IDP open-source, notamment Backstage, l'emportent sur les solutions entièrement développées en interne en termes de coût total de possession (TCO) et de time-to-value, pour une équipe de taille réduite.

Résultats

H1, validée avec nuance. Le Lead Time for Changes et le Change Failure Rate sont les deux KPIs les plus sensibles pour mesurer la valeur d'une IDP, confirmation apportée par notre interview terrain (podcast, 00:47:36). La réduction de friction est réelle lorsque l'IDP expose des golden paths et supprime les validations manuelles. Elle reste conditionnée à un taux d'adoption effectif supérieur à 40 % de DAU/MAU.

H2, partiellement validée. L'adoption résulte d'un équilibre entre facteurs culturels et technologiques (podcast, 00:48:52). Un champion interne est nécessaire mais pas suffisant : la technologie doit correspondre au contexte projet et aux pratiques existantes de l'équipe.

H3, conditionnellement validée. Backstage convient aux startups et aux PME. Les grandes entreprises ont davantage intérêt à investir dans une solution custom (podcast, 00:49:26). Notre benchmark confirme un avantage Backstage sur le time-to-value, et un désavantage sur la complexité initiale de mise en place.

Recommandation

Étant Lead Developer, nous déploierions Backstage en mode *Software Catalog* seul avec un time-to-value inférieur à deux semaines, sans plugin custom avant trois mois d'adoption prouvée (NPS interne ≥ 7 , DAU/MAU $\geq 40\%$). Nous ne déploierions pas une IDP full-featured dès le départ. Le risque de *platform tax* (l'IDP qui ralentit plus qu'elle n'accélère) est le risque numéro un identifié tout au long de cette analyse.

Contre-arguments

Trois objections sérieuses traversent l'ensemble de ce mémoire. Pour chacune, nous assumons une réponse et précisons ce qui nous ferait changer d'avis (la condition de falsification).

OBJ 1 : « Un bon CI/CD suffit, l'IDP c'est de l'over-engineering »

L'objection. Des pipelines CI/CD bien configurés (GitHub Actions, GitLab CI) couvrent déjà l'essentiel des besoins d'automatisation. Ajouter une couche IDP par-dessus multiplie la complexité sans apporter de valeur proportionnelle. Pourquoi investir dans une plateforme interne si les outils existants font le travail ?

Notre réponse. La distinction est fondamentale : le CI/CD automatise l'exécution d'un pipeline défini, mais ne dit rien sur qui le définit, comment il est maintenu, ni comment une nouvelle équipe le découvre. Sans IDP, chaque équipe réinvente son pipeline, ce qui crée une variance coûteuse, un onboarding lent, et une dette d'infrastructure cachée qui s'accumule silencieusement. L'IDP n'est pas un doublon du CI/CD : c'est la couche qui rend le CI/CD reproductible, documenté et self-service à l'échelle de l'organisation.

Ce qui nous ferait changer d'avis. Des données montrant que des organisations avec un CI/CD mature mais sans IDP atteignent les mêmes scores DORA « elite » que celles avec IDP, sur un échantillon de plus de 100 organisations comparables. Ces données n'existent pas à ce jour dans la littérature disponible.

OBJ 2 : « Backstage est trop lourd pour une petite équipe »

L'objection. Backstage requiert une équipe dédiée pour le maintenir (plugins, mises à jour, gouvernance du Software Catalog). Pour une équipe réduite de moins de 10 développeurs, c'est du temps pris directement sur le produit, sans retour visible à court terme.

Notre réponse. Le Software Catalog seul apporte de la valeur dès trois à cinq équipes, sur deux axes immédiats : visibilité sur les services existants et onboarding accéléré. Les plugins community réduisent l'effort de build. Surtout, notre benchmark montre qu'une solution custom n'est pas gratuite non plus : elle a un coût invisible constitué du temps de développeur senior non comptabilisé dans les estimations initiales. La question n'est pas « Backstage ou rien » : c'est « Backstage ou custom ». Sur le TCO à deux ans, Backstage gagne pour une équipe réduite.

Ce qui nous ferait changer d'avis. Mesurer sur notre propre benchmark que le setup et la maintenance de Backstage dépassent 0,5 ETP par mois pour une équipe de cinq ingénieurs, sur une période de trois mois.

OBJ 3 : « L'IDP devient une bureaucratie déguisée »

L'objection. En voulant standardiser, on crée inévitablement des gates obligatoires, des formulaires, des approbations. L'IDP finit par ralentir plus qu'elle n'accélère, et personne ne l'utilise vraiment. C'est ce qu'on appelle la platform tax : la plateforme censée simplifier le travail devient elle-même une friction.

Notre réponse. La charge bureaucratique vient d'une IDP qui impose des workflows non-optionnels. Une IDP orientée pave the road (et non block the road) expose des capacités self-service que les équipes adoptent parce que c'est plus rapide que leurs alternatives, pas parce qu'elles y sont contraintes. La différence de conception est radicale : dans un cas la plateforme est un passage obligé, dans l'autre c'est le chemin de moindre résistance.

Ce qui nous ferait changer d'avis. Un NPS interne inférieur à 6 sur 10 après trois mois d'adoption, un Lead Time qui augmente après déploiement de l'IDP dans notre mesure, ou des données montrant que plus de 50 % des IDPs déployées sont abandonnées dans les douze mois suivant leur mise en production.

Tableaux de validation

Table des hypothèses

Critère	H1, réduction friction dev→prod	H2, adoption et culture organisationnelle	H3, Backstage vs. custom (TCO & time-to-value)
Hypothèse	Une IDP bien adoptée réduit la friction dev→prod.	L'adoption dépend de la culture org (silos vs collaboration, champion interne).	Backstage open-source bat le custom en TCO et time-to-value pour équipe réduite.
Indicateur mesurable	Lead Time for Changes (LTfC), MTTR, Change Failure Rate (CFR), DORA 4 keys.	Taux d'adoption actif (DAU/MAU), NPS interne (0-10), présence/absence de champion identifié.	Time-to-setup (heures), effort maintenance (h/semaine), communauté (stars, contributeurs actifs), coût infra mensuel.
Protocole de test	Extraire les métriques git (commit vers deploy) + incident log sur un projet pilote. Comparer 2 périodes (avant/après IDP) ou 2 équipes (avec/sans IDP). Outil : <code>measure_dora.sh</code> + GitHub API.	5 entretiens semi-directifs (2 devs, 1 ops, 1 PM, 1 lead). Grille d'analyse culture Westrum. Podcast comme source qualitative principale.	Déployer Backstage (Docker) + portail custom minimal (React + YAML). Mesurer setup time chronométré, temps d'ajout d'un composant, coût infra. Comparer GitHub stats à date fixe.
Résultat	Validée avec nuance : LTfC corrélé à l'adoption des golden paths ; données podcast renforcent le signal.	Partiellement validée : culture nécessaire mais pas suffisante (podcast, 00:48:22).	Conditionnellement validée : avantage Backstage pour PME, custom pertinent à grande échelle.
Niveau de confiance	Moyen N faible (1 projet pilote), biais temporel, biais de contexte unique. Podcast : 1 expert, non généralisable.	Faible-Moyen Biais déclaratif, N=1 expert externe, pas de mesure quantitative sur notre propre org.	Moyen Contexte d'équipe réduite (< 10 devs), résultats non généralisables aux grandes orgs. Biais source Humanitec.

Table des claims → sources

Claim (affirmation)	Source primaire	Pourquoi crédible	Limites / biais
90 % des développeurs perdent 6 h ou plus par semaine à cause de frictions organisationnelles	Atlassian, State of Developer Experience 2025, 29 juillet 2025 ²	Large échantillon, rapport annuel de référence sur l'expérience développeur	Auto-déclaratif, biais de sélection, contexte majoritairement anglophone
Les développeurs ne consacrent que 16 % de leur temps à coder	Atlassian, State of Developer Experience 2025, 29 juillet 2025 ²	Même source, données cohérentes avec d'autres rapports sectoriels	Même limites : mesure déclarative, non vérifiable objectivement
Les investissements dans les infrastructures cloud ont augmenté de 29 % en un an sur le dernier trimestre 2025	Omdia, Global Cloud Infrastructure Spending, 26 mars 2026 ¹	Cabinet de recherche indépendant, données marché trimestrielles	Agrégat global, ne reflète pas nécessairement les décisions des équipes produit individuelles
Les DORA metrics (LTfC, DF, MTTR, CFR) sont les indicateurs standards de performance DevOps	DORA State of DevOps Report 2024, Google Cloud / DORA Research ³	Étude longitudinale, +39 000 répondants, 10 ans de données, approche peer-reviewed	Auto-déclaratif, biais de sélection (volontaires), majorité US
Les « elite performers » DORA déploient >1 000× plus fréquemment avec 3× moins de failures	DORA State of DevOps Report 2023, pp. 12 à 15 ⁴	Segmentation rigoureuse par niveau de performance, même corpus longitudinal	Corrélation différente de causalité : facteurs confondants (taille org, secteur, maturité)
Une Platform Team qui expose des self-service capabilities réduit le cognitive load dev	Skelton & Pais, Team Topologies (2019, IT Revolution), ch. 5 « Platform Teams » ^{5 6}	Référence académique et industry, auteurs actifs, cité dans 500+ organisations	Prescriptif, focus grandes orgs, peu de données quantitatives
Backstage est le standard open-source de facto pour les IDP	CNCf Landscape + repo backstage/backstage GitHub (Apache 2.0) ^{7 8}	Projet CNCf Incubating, +33 000 stars, +1 900 contributeurs, adopté par Spotify, Netflix, Airbnb	Peut être surdimensionné pour équipe < 5 devs, courbe d'apprentissage réelle

Claim (affirmation)	Source primaire	Pourquoi crédible	Limites / biais
Le platform engineering est une priorité DevOps 2024 à 2025	Gartner, Hype Cycle for Platform Engineering 2023 ⁹	Analyste de référence, méthodologie structurée, non affilié aux vendeurs	Paywall partiel, biais Gartner vers grandes entreprises
Les IDP custom coûtent 2 à 3× plus à maintenir vs. open-source sur 2 ans	Humanitec, State of Platform Engineering Vol. 2 (2023), N=500 ¹⁰	Large échantillon, segmenté par taille org	Biais fort : Humanitec est éditeur IDP concurrent, chiffre à croiser avec source neutre
Le Developer Experience (DevEx) est corrélé à la rétention et la productivité	Forsgren et al., « SPACE: A Framework for Understanding Developer Productivity » (2021, ACM Queue) ¹¹	Publication peer-reviewed ACM, auteurs DORA, cadre adopté par GitHub/Microsoft	Difficile à isoler (multi-facteurs), pas de valeur seuil universelle
Lead time et failure rate sont les deux KPI essentiels pour mesurer la valeur d'une IDP	Podcast, Arnaud Pradeilles, consultant senior ex-TF1, 29/05/2026, 00:47:36 ¹²	Praticien terrain, expérience large sur des organisations multi-secteurs	Source unique (1 expert), contexte grandes orgs, non peer-reviewed
Backstage convient aux PME ; les grandes entreprises ont intérêt à faire du custom	Podcast, Arnaud Pradeilles, 29/05/2026, 00:49:26 ¹²	Même source, cohérent avec Gartner (contexte-dépendant)	Même limites : opinion d'un expert, pas une étude de marché

Hypothèse 1 : une IDP bien conçue réduit-elle vraiment la friction dev→prod ?

Annonce et cadrage

L'hypothèse H1 postule qu'une Internal Developer Platform bien adoptée réduit la friction du développement jusqu'à sa mise en production, de manière mesurable via les métriques DORA, en particulier le Lead Time for Changes, le Change Failure Rate et le MTTR. Il ne s'agit pas d'une affirmation absolue : la réduction de friction dépend de la qualité de conception de l'IDP et, surtout, de son taux d'adoption réel par les équipes.

Pour évaluer cette hypothèse, il faut d'abord comprendre par quel mécanisme une IDP agit sur ces métriques, avant d'examiner les preuves disponibles et leurs limites.

I. Arguments

La friction organisationnelle est le premier levier d'amélioration disponible

Avant même de parler d'IDP, le contexte justifie l'investissement. Selon le rapport State of Developer Experience 2025 d'Atlassian, 90 % des développeurs perdent six heures ou plus par semaine à cause de frictions organisationnelles (changements de contexte entre outils, recherche d'information, compréhension de processus internes) ². La moitié en perd plus de dix heures. Les développeurs ne consacrent par ailleurs que 16 % de leur temps à coder. Les outils censés améliorer la productivité deviennent eux-mêmes une source de complexité.

C'est précisément le problème que l'IDP cherche à résoudre : en exposant des golden paths et des capacités self-service, elle supprime les allers-retours entre équipes, les validations manuelles et la redécouverte de processus internes à chaque nouveau projet.

Le lien causal passe par des mécanismes intermédiaires identifiables

L'IDP ne réduit pas le Lead Time directement, elle le fait via trois mécanismes intermédiaires qu'il faut distinguer pour rendre l'hypothèse testable.

- **Standardisation des pipelines CI/CD** : chaque équipe n'a plus à configurer Kubernetes ou ses pipelines from scratch. Un template standardisé réduit le temps de setup et les erreurs de configuration, deux sources majeures de Lead Time élevé.
- **Suppression des validations manuelles** : les gates de conformité et de sécurité, intégrés by default dans les golden paths, ne nécessitent plus d'intervention humaine à chaque déploiement.

- **Onboarding accéléré** : un développeur qui rejoint une équipe peut déployer en autonomie dès le premier jour, sans dépendre d'un senior pour comprendre l'infrastructure.

Ces mécanismes sont cohérents avec ce que Skelton et Pais décrivent dans Team Topologies (2019) : une Platform Team qui expose des self-service capabilities réduit le cognitive load des équipes produit, libérant de la bande passante pour la production de valeur métier ⁵.

Les données DORA confirment la corrélation à grande échelle

Le DORA State of DevOps Report 2023 montre que les organisations classées « elite performers » déploient plus de 1 000 fois plus fréquemment que les « low performers », avec trois fois moins d'échecs de déploiement ⁴. Sur neuf années de données et plus de 33 000 répondants, la corrélation entre pratiques d'automatisation et performance DevOps est robuste. Les IDPs, en tant qu'outils de standardisation et d'automatisation de ces pratiques, s'inscrivent directement dans cette logique.

Notre interview avec Arnaud Pradeilles, consultant senior ayant accompagné des organisations multi-secteurs, confirme ce signal terrain : il identifie le Lead Time for Changes et le Change Failure Rate comme les deux KPIs les plus sensibles pour mesurer la valeur d'une IDP (podcast, 00:47:36) ¹². Ce sont également les deux métriques les plus directement influencées par les mécanismes décrits ci-dessus.

II. Contre-arguments et limites

L'objection principale : un bon CI/CD suffit

On pourrait objecter qu'un outillage CI/CD mature couvre déjà l'essentiel de ces besoins sans nécessiter la complexité d'une IDP. La réponse tient à la distinction entre exécuter un pipeline défini et rendre ce pipeline reproductible, documenté et self-service à l'échelle. Ce point a été développé dans la section « Contre-arguments » générale ; il reste valide ici au niveau de H1 spécifiquement : si le CI/CD suffisait, on observerait des scores DORA « elite » chez des organisations sans IDP, ce que la littérature ne documente pas.

La limite méthodologique centrale : corrélation ≠ causalité

C'est la limite la plus sérieuse de H1. Les études DORA sont auto-déclaratives et observationnelles : elles mesurent une corrélation entre pratiques et performance, sans randomisation. Les facteurs confondants sont réels : les organisations qui investissent dans une IDP sont généralement plus matures, mieux dotées en budget et en profils seniors. Il est impossible de savoir, à partir de ces données seules, si c'est l'IDP qui améliore les métriques ou si ce sont les organisations déjà performantes qui déploient des IDPs.

Notre protocole de mesure avant/après sur un projet pilote unique n'échappe pas à ce biais temporel : la performance aurait peut-être progressé de toute façon. Le niveau de confiance reste donc **Moyen**.

La condition structurelle : l'adoption

H1 ne peut être validée que si l'IDP est réellement utilisée. En dessous d'un seuil d'environ 40 % de DAU/MAU, les équipes qui n'adoptent pas l'IDP conservent leurs anciens workflows, et le signal DORA est dilué au point d'être inexploitable. Ce n'est pas une nuance mineure : c'est une condition préalable à toute mesure. Une IDP déployée mais non adoptée ne peut pas réduire le Lead Time, quelle que soit la qualité de sa conception.

III. Recommandation finale, décision Lead Developer

Sur la base de ce que H1 nous apprend, voici la décision que nous prendrions en tant que Lead Developer.

Ce que nous ferions. Poser les quatre métriques DORA comme baseline dès le début du projet, avant tout déploiement d'IDP. Sans mesure initiale, il est impossible de démontrer quoi que ce soit six mois plus tard. Nous mesurerions en priorité le Lead Time for Changes et le Change Failure Rate, les deux KPIs les plus sensibles selon notre analyse et confirmés par notre interview terrain.

Ce que nous ne ferions pas. Déployer une IDP full-featured dès le départ. Le risque de platform tax est le risque numéro un identifié : une IDP qui impose des workflows avant d'avoir démontré sa valeur génère de la résistance, des contournements, et finit par être abandonnée. Nous commencerions par le Software Catalog seul (time-to-value inférieur à deux semaines) et n'étendrions les fonctionnalités qu'après avoir atteint 40 % de DAU/MAU et un NPS interne ≥ 7 .

La condition préalable non négociable. Identifier un champion interne avant tout déploiement. Sans portage humain de l'initiative, les golden paths ne seront pas utilisés, l'adoption ne décollera pas, et H1 ne pourra jamais être validée dans ce contexte, quel que soit le niveau d'investissement technique.

H1 est donc **validée avec nuance** : non pas parce que la preuve causale est irréfutable, mais parce que le faisceau d'indices convergents (littérature DORA, expertise terrain, mécanismes intermédiaires identifiables) est suffisamment solide pour justifier l'action, à condition de mesurer rigoureusement et de ne pas sur-déploier dès le départ.

Hypothèse 2 : l'adoption d'une IDP dépend-elle davantage de facteurs organisationnels et culturels que technologiques ?

Nous venons de voir qu'une IDP bien conçue permet de réduire la friction du début du développement d'une application jusqu'à sa mise en production. Mais un autre facteur important concerne les conditions pour qu'une IDP soit adoptée par une équipe de développeurs. Certaines entreprises ont déployé des IDPs sans obtenir les gains espérés, notamment à cause de facteurs non techniques. Dans cette partie, il s'agira d'analyser l'hypothèse que l'adoption d'une IDP dépend davantage de facteurs organisationnels et culturels que technologiques. Ces facteurs sont notamment la collaboration entre les équipes, la réduction des silos, ou encore l'existence ou non de champions internes capables de mettre en place cette transformation.

I. L'IDP comme réponse à un problème de complexité sociotechnique

On pense souvent que l'IDP n'est qu'une solution permettant le déploiement d'applications ou de simplifier la partie architecture. En réalité, les IDP sont apparues dans un contexte où la complexité de l'environnement d'un développeur ne cessait d'augmenter. Le cloud, la multitude d'architectures différentes, le DevOps, ont multiplié les outils et technologies auxquels les équipes de développement doivent faire face. Des outils comme Docker ou Kubernetes répondent à des besoins essentiels (automatisation, scalabilité, sécurité) et ont largement contribué à l'accélération du processus de développement. Néanmoins, leur accumulation rend le travail du développeur de plus en plus difficile. Par conséquent, le but d'une IDP n'est pas seulement de répondre à un problème technique : il y a aussi un enjeu organisationnel et culturel.

I.A. Une IDP vise à réduire la charge cognitive des équipes de développement

Durant les 50 dernières années, le rôle du développeur a bien évolué. Aujourd'hui, le développement moderne implique la conteneurisation Docker ou Kubernetes, les CI/CD, les différentes plateformes Cloud ou encore les outils de monitoring comme Grafana. Dans l'ouvrage *Team Topologies*, Matthew Skelton et Manuel Pais introduisent le concept de charge cognitive : « les équipes, comme les individus, ont une capacité cognitive limitée »⁵. Lorsque cette capacité est dépassée, les équipes rencontrent plusieurs difficultés :

- **Augmentation des changements de contexte** : ralentit l'équipe et l'empêche de se concentrer.
- **Manque de maîtrise** : ils doivent jongler entre plusieurs tâches sans pouvoir approfondir les sujets.
- **Perte de maîtrise sur leurs domaines de compétence** : entraîne des difficultés liées à l'évolution de leurs produits.

L'objectif d'une IDP n'est pas de remplacer les outils existants, mais de masquer une partie de leur complexité derrière des interfaces et des processus standardisés. De ce fait, les développeurs peuvent se concentrer sur ce qui importe réellement : le développement métier de leurs projets, sans avoir besoin de maîtriser l'ensemble des technologies sous-jacentes.

Par conséquent, la valeur d'une IDP ne réside pas uniquement dans les technologies qu'elle intègre, mais aussi dans sa capacité à réduire la charge cognitive des développeurs. Une plateforme techniquement complète mais difficile à utiliser risque d'être peu adoptée. À l'inverse, une plateforme qui simplifie réellement le travail quotidien apportera davantage de valeur, même si elle repose sur des technologies plus classiques.

I.B. Une IDP est un système sociotechnique avant d'être un outil

Si l'IDP permet de réduire la charge cognitive des développeurs, sa réussite ne dépend pas seulement des technologies qui la composent. Son efficacité repose aussi sur son intégration dans l'entreprise. Cette idée est illustrée par le concept de système sociotechnique, selon lequel les performances d'un projet technologique dépendent des interactions sociales, des individus et de l'organisation dans laquelle ils évoluent.

Cette approche est en accord avec le programme de recherche DORA : « pour améliorer une plateforme, les organisations doivent cesser de la considérer comme un projet technique et adopter une approche centrée sur l'utilisateur et sur le produit »¹³. Team Topologies va dans le même sens : une plateforme n'apporte de la valeur que si les interactions entre les équipes sont clairement définies⁶. Si les processus sont mal pensés ou les responsabilités mal établies, l'IDP deviendrait une source supplémentaire de friction et de complexité.

II. La structure organisationnelle conditionne fortement l'adoption

II.A. Les silos organisationnels sont une source majeure de friction

L'une des aspirations principales des IDP est de réduire les frictions entre les équipes. En revanche, dans de nombreuses entreprises, les équipes (développement, Ops, sécurité) fonctionnent de manière isolée : c'est ce qu'on appelle les silos organisationnels. Cette fragmentation est une source de friction sur l'adoption d'une IDP.

Tout d'abord, les délais de validation sont significativement allongés. Lorsqu'un déploiement nécessite l'intervention successive de plusieurs équipes, le processus perd en fluidité, un sujet évoqué par Arnaud Pradeilles dans le podcast¹² et que l'on retrouve dans de nombreuses entreprises. De plus, le manque de visibilité entre les équipes empêche l'adoption de standards unifiés. Chaque équipe peut conserver ses propres pratiques, ce qui va à l'encontre de l'objectif de standardisation d'une IDP. En outre, si l'IDP est trop rigide ou contraignante, les développeurs peuvent revenir à leurs anciennes pratiques, même si la plateforme est performante.

II.B. Le modèle « Platform Team » favorise l'appropriation de l'IDP

Certaines entreprises contournent ces limites en adoptant le modèle Platform Team : l'IDP n'est plus considérée comme un projet technique mais comme un produit interne. Dans cette approche, les développeurs sont les « clients » de la plateforme, qui s'adapte à leurs besoins via des feedbacks continus. L'équipe plateforme a pour objectif d'intégrer ces retours et de répondre à leurs besoins. La plateforme reste ainsi en constante évolution.

Le programme de recherche DORA insiste sur la notion de Developer Experience comme indicateur central : « pour garantir la réussite de la mise en œuvre, il est nécessaire de désigner un chef de produit chargé de se concentrer sur l'expérience des développeurs (DevEx) et d'élaborer une feuille de route stratégique fondée sur une compréhension approfondie des besoins des développeurs »¹³.

III. L'adoption dépend principalement de la culture et de l'accompagnement humain

III.A. Le rôle du champion interne dans la diffusion de l'IDP

Dans son ouvrage Diffusion of Innovations, le sociologue Everett Rogers démontre que l'adoption d'une nouveauté est avant tout un processus social qui se fait « au fil du temps »¹⁴. Certains individus jouent un rôle déterminant en influençant leurs pairs et en facilitant l'acceptation du changement : les champions internes.

Appliquée au Platform Engineering, cette notion fait référence à un développeur expérimenté (lead développeur, tech lead) ou à un membre de l'équipe Platform Engineering. Son rôle ne consiste pas uniquement à présenter les fonctionnalités de la plateforme : il accompagne concrètement les équipes, organise des formations, facilite l'onboarding des nouveaux développeurs et recueille leurs retours. À l'inverse, l'absence de ce relais en interne peut faire considérablement défaut à une IDP, même si la plateforme est techniquement bien construite.

III.B. L'adoption dépend de la perception de valeur par les développeurs

Le degré d'adoption d'une IDP est directement relié à la valeur qu'elle est capable de produire, perçue par les développeurs dans leurs activités quotidiennes. Cette valeur perçue repose sur plusieurs éléments : le gain de temps de déploiement, l'optimisation des workflows, et la réduction du nombre de tickets vers les équipes d'infrastructure.

Inversement, une IDP peut être contournée si elle apparaît comme une source de contraintes. Des processus trop lourds, une complexité supplémentaire ou une perte de flexibilité peuvent amener les développeurs à privilégier leurs propres outils. L'adoption de l'IDP repose donc en premier lieu sur sa capacité à satisfaire les attentes pragmatiques des développeurs au quotidien.

Recommandation, décision Lead Developer

Ce que nous ferions. Identifier et missionner un champion interne avant tout déploiement, organiser des entretiens réguliers avec les équipes pour mesurer la perception de valeur, et adopter le modèle Platform Team pour traiter l'IDP comme un produit interne plutôt qu'un projet technique.

Ce que nous ne ferions pas. Déployer une IDP sans étude préalable de la structure organisationnelle (silos, répartition des responsabilités). Une plateforme techniquement excellente mais imposée sans accompagnement humain reste sous-utilisée, quelle que soit sa qualité.

H2 est donc **partiellement validée** : la culture organisationnelle est une condition nécessaire mais non suffisante. La technologie doit correspondre au contexte projet et aux pratiques existantes de l'équipe, car ni l'aspect humain ni l'aspect technique ne suffisent isolément.

Hypothèse 3 : les IDP open-source (Backstage) l'emportent-elles sur le custom en TCO et time-to-value pour une équipe réduite ?

Pour une équipe de taille réduite qui veut intégrer une IDP, le choix se fait souvent entre deux approches : construire une IDP custom ou partir d'une solution open-source comme Backstage, le framework pour portails de développeur créé par Spotify. Ce que l'on considère comme custom ici est toute solution développée et maintenue en interne, qu'il s'agisse d'un outil construit à partir de zéro ou d'un assemblage de briques internes (scripts, CLI, dashboards) sans framework IDP existant.

Cette question cache un vrai coût, mesuré via deux notions : le TCO (Total Cost of Ownership), qui représente le coût total d'une solution (pas seulement les frais cloud mais aussi le temps passé à construire, maintenir et faire évoluer la plateforme), et le time-to-value, c'est-à-dire le temps nécessaire avant que la solution apporte un bénéfice réel et quotidien. Cette hypothèse se limite volontairement à une équipe réduite (moins de 10 développeurs), seuil au-delà duquel le coût et l'adoption d'une IDP changent complètement.

Indicateurs et protocole de validation

Pour comparer Backstage et une IDP custom de façon concrète, on retient quatre indicateurs :

- **Time-to-setup** : temps pour avoir une première version utilisable, déploiement de Backstage via Docker vs. construction d'un portail minimal (React + YAML), jusqu'à l'affichage d'un catalogue de services basique.
- **Effort de maintenance** : temps passé chaque semaine à faire vivre la solution (mises à jour, plugins, bugs), complété par des retours d'expérience d'équipes ayant vécu cette charge sur plusieurs années (Paddle, PagerDuty).
- **Communauté** : nombre de stars et de contributeurs actifs sur le repo GitHub de Backstage, pour anticiper la maintenance future.
- **Coût infra mensuel** : facture cloud brute pour faire tourner chaque solution.

Le protocole consiste à déployer les deux solutions en parallèle sur un projet pilote, mesurer chaque indicateur sur une période d'un mois, puis croiser ces résultats avec les cas réels (Paddle, PagerDuty) pour vérifier si nos mesures à petite échelle confirment ou contredisent les retours d'expérience à plus grande échelle.

I. Backstage : un écosystème mature qui apporte de la valeur vite

I.A. Un outil open-source porté par une grande communauté

Le premier avantage de Backstage n'est pas technique mais collectif. Quand une équipe choisit une IDP custom, elle est seule à gérer chaque ligne de code, chaque correctif et chaque nouvelle fonctionnalité. Quand elle choisit Backstage, elle s'appuie sur une communauté déjà active. Les chiffres du repo officiel le montrent : projet CNCF avec plus de 33 000 stars et plus de 1 900 contributeurs, utilisé par Spotify, Netflix et Airbnb ⁷. Cela a un effet direct sur le TCO : une grande partie des fonctionnalités (catalogue, documentation, templates, plugins) est développée et maintenue par la communauté, et pas par l'équipe qui adopte l'outil.

L'enquête annuelle de la CNCF ⁸ confirme que le platform engineering et les portails développeur sont passés d'une simple tendance à une pratique courante dans les entreprises cloud-native. Choisir Backstage, c'est choisir un outil que le marché valide, là où le custom signifie partir seul.

I.B. Un time-to-value court grâce au catalogue de services

Le Software Catalog (inventaire centralisé des services, de leurs propriétaires et de leur documentation) apporte déjà de la visibilité et accélère l'onboarding dès les premières équipes, sans déployer une plateforme complète. Là où une IDP custom doit être pensée, conçue et codée from scratch, Backstage propose dès l'installation une base fonctionnelle : on génère une instance, on connecte ses repos, on déclare ses services dans des fichiers catalog-info.yaml, et le catalogue est prêt. Le time-to-value se compte alors en jours, pas en mois.

II. Le vrai coût du custom : un TCO souvent sous-estimé

II.A. Le piège du « fait maison » : une maintenance cachée

L'argument le plus fréquent en faveur du custom est qu'il serait « gratuit » et parfaitement adapté aux besoins de l'équipe. C'est une illusion : le custom n'a pas de coût de licence, mais il a un coût en temps d'ingénieurs, souvent invisible car il n'apparaît sur aucune facture. Plusieurs retours d'expérience l'illustrent. Paddle a maintenu pendant plus de quatre ans un Backstage auto-hébergé avec un backend Go entièrement custom, avant de migrer vers une solution managée en 2024 à cause de la charge de maintenance ¹⁵. PagerDuty a suivi le même chemin ¹⁶. Roadie estime qu'un Backstage auto-hébergé « gratuit et open-source » demande en réalité entre 3 et 12 ingénieurs dédiés selon la taille ¹⁰ (chiffre à prendre avec prudence, Roadie vend une solution managée, mais dont l'ordre de grandeur reste un signal fort).

Les principaux postes de ce coût caché sont : le temps de construction de départ, la maintenance régulière (mises à jour, correctifs de sécurité, gestion des dépendances), les évolutions fonctionnelles, et le risque lié au départ de personnes-clés qui portent le savoir sur le custom.

II.B. Backstage vs. custom : le TCO sur la durée

Poste de coût	Backstage open-source	IDP custom
Coût de construction de départ	Faible (base fournie)	Élevé (tout est à développer)
Time-to-value	Court (jours)	Long (semaines à mois)
Maintenance régulière	Partagée avec la communauté	À la charge de l'équipe
Personnalisation	Élevée (plugins)	Maximale
Soutien extérieur	Grande communauté active	Aucun
Risque de dépendre d'une personne	Faible	Élevé

La lecture du tableau penche clairement pour Backstage dans le cas d'une équipe réduite. Le custom ne devient intéressant que si l'équipe a un besoin tellement spécifique qu'aucune brique de la communauté n'y répond, ce qui reste rare à petite échelle.

III. Quand l'avantage de Backstage se nuance

III.A. Backstage n'est pas magique : le coût caché du « day 2 »

Daniel Bryant, consultant technique, explique lors de PlatformCon 2024 que le catalogue de services de Backstage est très bon, mais que son support des opérations « day 2 » (maintenance, vie de la plateforme une fois en production) l'est beaucoup moins ¹⁷. Selon lui, une approche centrée sur le développeur offre une très bonne expérience au démarrage, mais devient souvent une façade difficile à tenir dans le temps. L'avantage de Backstage est donc réel au lancement, mais moins évident sur la durée.

III.B. Un custom bien maîtrisé peut durer si le périmètre est limité

Le cas d'Uber est parlant : son outil interne SLATE (environnements de test temporaires à durée de vie limitée) est un outil de plateforme entièrement custom, en production depuis plusieurs années, qui continue d'évoluer ¹⁸. Mais ce contre-exemple appelle une nuance importante : SLATE couvre un périmètre bien plus limité que Backstage (le test uniquement). Ce que ce cas montre, c'est que le custom peut durer quand son périmètre est volontairement limité et bien maîtrisé. Daniel Bryant parle d'un « missing middle » entre le portail (type Backstage) et l'infrastructure (type Terraform), comblé par des orchestrateurs comme Crossplane ou Kratix ¹⁷.

Apport du podcast

L'entretien avec Arnaud Pradeilles vient appuyer la nuance qui ressort des contre-arguments. Sur le choix entre open-source et custom, il considère que Backstage convient aux startups et aux PME, alors que les grandes entreprises ont plus à gagner avec une solution custom (podcast, 00:49:26) ¹². Ce retour de terrain colle bien avec ce qu'on tire des sources écrites : la réponse n'est pas universelle, elle dépend de la taille et de la maturité de l'entreprise. La limite est que cela reste l'avis d'un seul expert, pas une étude de marché.

Recommandation, décision Lead Developer

Pour une équipe réduite, nous déploierions Backstage plutôt qu'une IDP custom, parce que sa communauté fait avancer l'outil vite et partage la maintenance. Les cas de Paddle et PagerDuty montrent que même des équipes capables de construire du custom finissent par y renoncer à cause de la charge de maintenance.

Ce que nous ferions :

- Déployer Backstage en mode « catalogue de services + documentation » seul, pour viser un time-to-value de moins de 2 semaines.
- Ne pas développer de plugin custom avant d'avoir prouvé l'adoption sur plusieurs mois.
- Mesurer dès le départ l'effort de maintenance réel.

Ce que nous ne ferions pas :

- Déployer une IDP avec toutes les fonctionnalités dès le départ (risque de platform tax).
- Partir sur du custom complet pour une équipe réduite, sauf besoin précis non couvert par la communauté et sur un périmètre volontairement limité (sur le modèle de SLATE).

H3 est donc **conditionnellement validée** : Backstage l'emporte pour une équipe réduite, pas parce qu'il serait meilleur sur tous les plans, mais parce qu'il met le coût là où une petite équipe peut le supporter : la configuration plutôt que la construction, et la communauté plutôt que l'équipe seule.

Conclusion

Au départ de ce mémoire, une question : comment construire une IDP qui réduit vraiment la friction du développement jusqu'à la mise en production, sans devenir une charge bureaucratique ? Les trois hypothèses testées permettent d'y répondre, non pas séparément, mais ensemble.

H1 a montré qu'une IDP bien adoptée réduit effectivement la friction dev→prod, de façon mesurable via les métriques DORA, en particulier le Lead Time for Changes et le Change Failure Rate. Mais cette réduction n'est ni automatique ni gratuite : elle n'apparaît qu'au-delà d'un certain seuil d'adoption (environ 40 % de DAU/MAU). Une IDP déployée mais non utilisée ne réduit rien.

H2 a expliqué pourquoi cette adoption est si déterminante : elle dépend davantage de facteurs organisationnels et humains (réduction des silos, présence d'un champion interne, perception de valeur par les développeurs). Une IDP techniquement parfaite mais mal portée en interne reste sous-utilisée.

H3 a tranché la question du « comment construire » pour une équipe réduite : partir d'une solution open-source comme Backstage plutôt que de tout développer en interne, parce que le coût et le risque sont partagés avec une communauté, et que le time-to-value est bien plus court.

Mis bout à bout, ces trois résultats dessinent une même réponse à la problématique. Ce qui transforme une IDP en charge bureaucratique, ce n'est pas l'outil, c'est la manière de le déployer. Une IDP qui impose des workflows obligatoires (*block the road*) génère de la résistance et finit contournée. Une IDP qui expose des capacités self-service que les équipes adoptent parce que c'est plus rapide (*pave the road*) réduit réellement la friction. La différence ne se joue pas sur la technologie, mais sur la conception et l'accompagnement.

Décision

Étant Lead Developer, nous déploierions Backstage en mode *Software Catalog + documentation* seul, pour un time-to-value inférieur à deux semaines, et nous n'étendrions les fonctionnalités qu'une fois l'adoption prouvée. Nous ne déploierions pas une IDP complète dès le départ : le risque de *platform tax* (la plateforme qui ralentit plus qu'elle n'accélère) est le risque numéro un que cette analyse a identifié de bout en bout.

Plan de suite et critères de décision

- **Avant tout déploiement** : identifier un champion interne (condition non négociable issue de H2) et poser les quatre métriques DORA comme baseline. Sans mesure initiale, aucune validation n'est possible plus tard.
- **À 3 mois** : évaluer l'adoption réelle. Aller de l'avant si NPS interne ≥ 7 et DAU/MAU $\geq 40\%$. En dessous, revoir l'accompagnement avant d'ajouter quoi que ce soit.
- **Avant d'ajouter un plugin custom** : vérifier qu'aucune brique communautaire ne répond au besoin, et que l'effort de maintenance reste sous 0,5 ETP/mois pour une équipe de cinq.
- **Prochaine étape de validation** : déployer Backstage sur un vrai projet de trois équipes pendant trois mois, avec mesure DORA avant/après, pour confronter nos hypothèses au terrain.

Vocabulaire

Charge cognitive : quantité d'informations et de connaissances qu'une équipe est capable de traiter efficacement.

Internal Developer Platform (IDP) : plateforme construite par une entreprise pour permettre aux développeurs de créer, déployer et exploiter des applications plus facilement, sans devoir gérer toute la complexité de l'infrastructure.

Platform engineering : discipline qui consiste à construire et maintenir des plateformes pour améliorer la productivité des développeurs.

Système sociotechnique : système selon lequel les performances d'un projet dépendent des interactions sociales, des individus et de l'organisation dans laquelle ils évoluent.

Silo organisationnel / travail en silo : mode d'organisation dans lequel chaque département ou service fonctionne de manière isolée, sans partage d'informations ni coordination transversale avec les autres services.

Golden path : chemin standardisé et documenté proposé par une IDP pour réaliser une tâche courante (créer un service, déployer une application), conçu pour être plus rapide à suivre que de tout configurer soi-même.

Platform tax : coût caché, en temps ou en friction, qu'une plateforme interne impose aux équipes qui doivent l'utiliser, par exemple lorsqu'elle ralentit les déploiements au lieu de les accélérer.

Pave the road / block the road : deux philosophies de conception d'une IDP. « Pave the road » expose des capacités self-service que les équipes adoptent parce qu'elles sont plus rapides que les alternatives. « Block the road » impose des workflows obligatoires, ce qui génère des contournements.

DAU/MAU : Daily Active Users / Monthly Active Users, ratio mesurant la part d'utilisateurs mensuels qui utilisent activement un outil au quotidien ; un indicateur courant d'adoption réelle.

NPS (Net Promoter Score) : indicateur de satisfaction, noté de 0 à 10, mesurant la probabilité qu'un utilisateur recommande un outil ou un service.

TCO (Total Cost of Ownership) : coût total d'une solution sur sa durée de vie, incluant non seulement les coûts directs (infrastructure, licences) mais aussi le temps d'ingénierie nécessaire à sa construction, sa maintenance et son évolution.

Time-to-value : délai entre le début d'un projet et le moment où la solution produit un bénéfice réel et mesurable pour ses utilisateurs.

ETP (Équivalent Temps Plein) : unité de mesure d'une charge de travail exprimée en nombre de personnes à temps plein ; par exemple, 0,5 ETP correspond à la moitié du temps de travail d'une personne à temps plein.

Day 2 (opérations) : ensemble des tâches liées à la vie d'une plateforme après sa mise en place initiale (maintenance, montées de version, gestion du cycle de vie), par opposition au « day 1 » (déploiement initial).

Sources

1. Omdia, *Global Cloud Infrastructure Spending Rose 29% in Q4 2025 as Hyperscalers Scaled AI Infrastructure Investment*, 26 mars 2026. [ondia.tech.informa.com](https://www.ondia.tech/informa.com)
2. Atlassian, *State of Developer Experience 2025*, 29 juillet 2025. atlassian.com
3. Google Cloud / DORA Research, *State of DevOps Report 2024* (DORA 4 keys : LTfC, DF, MTTR, CFR). dora.dev
4. Google Cloud / DORA Research, *State of DevOps Report 2023*, pp. 12 à 15 (elite performers).
5. Team Topologies, *Newsletter (July 2024): Mastering Team Effectiveness, The Power of Managing Cognitive Load*, Ana Ciobotaru, juillet 2024. teamtopologies.com
6. Team Topologies, *Key Concepts* (documentation technique). teamtopologies.com
7. Backstage, dépôt officiel *backstage/backstage* (licence Apache 2.0). github.com/backstage/backstage
8. CNCF, *Annual Survey Report*. cncf.io
9. Gartner, *Hype Cycle for Platform Engineering 2023*.
10. Humanitec, *State of Platform Engineering Vol. 2* (2023, N=500) ; chiffre croisé avec Roadie, *Backstage: How Much Does It Really Cost?*, estimation de 3 à 12 ingénieurs pour un Backstage self-hosted. roadie.io
11. Forsgren et al., *SPACE: A Framework for Understanding Developer Productivity* (2021, ACM Queue). queue.acm.org
12. Podcast, Arnaud Pradeilles, consultant senior ex-TF1, 29/05/2026 (00:47:36 ; 00:48:22 ; 00:48:52 ; 00:49:26).
13. Google Cloud, *2025 DORA State of AI-assisted Software Development Report*, 2025. dora.dev
14. Wikipedia, *Diffusion of Innovations*. en.wikipedia.org
15. Roadie, *From Self-Hosted Backstage to Roadie* (cas Paddle). roadie.io
16. Page It to the Limit (PagerDuty), *IDPs with Dave Bresci*. pageittothelimit.com
17. Daniel Bryant, *Platform Orchestrators: The Missing Middle of Internal Developer Platforms*, PlatformCon 2024. speakerdeck.com, [vidéo YouTube](https://www.youtube.com/watch?v=...)
18. Uber Engineering Blog, *Simplifying Developer Testing Through SLATE*. uber.com